

STACK vs HEAP MEMORY

A 2-3 MINUTE ENGINEERING CHEATSHEET FOR SENIOR SDE / TECH LEAD INTERVIEWS

1) WHY

- Understand lifetime, ownership and when memory is released.
- Reason about allocation cost, locality and performance.
- Avoid memory failures: stack overflow, OOM, fragmentation, leaks, use-after-free.
- Design for recursion limits, concurrency and reliability.

Stack

- Fast, local, predictable
- Automatic lifetime
- Bounded size (per thread)
- LIFO (last-in, first-out)

Heap

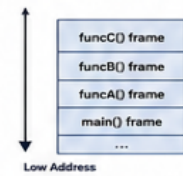
- Flexible, shareable
- Manual/RAII/GC lifetime
- Potentially large
- Non-LIFO

2) CORE IDEA

STACK

Per-thread call stack made of frames. Automatic storage duration. LIFO: created on call, destroyed on return.

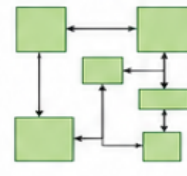
Thread / Call Stack (LIFO)



HEAP

Dynamic storage from the heap. Flexible lifetime independent of call stack. Managed manually, via RAI, reference counting, or Garbage Collector.

Heap (dynamic)



Values vs. References: Languages differ.

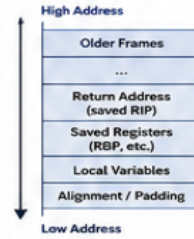
C/C++: values live in stack frames; pointers may point to heap.

Java/C#: object references on stack; objects on heap.

Rust: values on stack by default; Box<T> on heap.

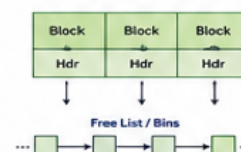
3) INTERNALS

STACK (per thread)



- Stack Pointer (SP) moves down (x86) / up (some arch).
- Call pushes a new frame; return pops it.
- Guard page at end to detect overflow.
- Typically 1MB-16MB per thread (OS / runtime configurable).

HEAP (process wide)



- Allocator manages blocks with metadata (header). Free lists / size classes / arenas / pages.
- Fragmentation: internal (waste inside block) and external (holes).
- Escape analysis may promote to stack.
- Backed by virtual memory (demand paging).
- Languages may add object headers (type, GC mark bits, length...).

4) COMPLEXITY

Aspect	STACK	HEAP
Allocation	Pointer bump of SP (typically O(1))	Allocator dependent; avg near O(1), may require synchronization / search (size classes, bins)
Deallocation	Pop frame: O(1)	free/delete; coalescing may be O(1)-O(n)
Cost Sources	Very low, contiguous	Metadata, locks/atomics, fragmentation, cache misses
GC / Refcount	N/A	GC: mark/sweep/compact costs; Refcount: atomic inc/dec, cycles
Locality	Excellent (contiguous frames)	Depends on allocator; possible scattered
Size Bound	Bounded (e.g., 1-16MB per thread)	Bounded by available virtual memory / limits
Failure Mode	Stack Overflow	Out Of Memory (OOM), Fragmentation

5) TRADE-OFFS

STACK PROS

- ✓ Very fast allocation and deallocation
- ✓ Cache friendly
- ✓ Deterministic lifetime
- ✓ No fragmentation
- ✓ Simple mental model

HEAP PROS

- ✓ Flexible lifetime
- ✓ Shareable (across functions/threads)
- ✓ Large allocations
- ✓ Data survives caller return
- ✓ Good for variable sized data

STACK CONS

- ✗ Limited size
- ✗ Lifetime tied to call
- ✗ Not shareable across async boundaries
- ✗ Deep recursion risk
- ✗ Large objects can overflow stack

HEAP CONS

- ✗ Slower, more overhead
- ✗ Fragmentation
- ✗ Memory leaks / UAF
- ✗ GC pauses or refcount overhead
- ✗ Poor locality (possible)

Use stack for small, short-lived, call-local data.
Use heap for large, long-lived, or shared data.

6) DISCUSSION

- When would you choose stack vs. heap? Give examples.
- How do value types vs. reference/pointer types affect placement?
- What is escape analysis and stack allocation promotion?
- What causes stack overflow? How is it detected?
- What is heap fragmentation? How to mitigate it?
- How do thread stacks work? Default sizes? Tuning?
- How do malloc/new work internally (high level)?
- How does a GC manage the heap? Pros/cons?
- What is use-after-free? How can you prevent it?
- How do you design allocation-free hot paths?
- How would you profile allocations and memory?

7) RELATED

- Call Stack: Sequence of function calls and frames.
- Allocators: malloc/new, jemalloc, tcmalloc, mimalloc.
- Garbage Collection: Mark & sweep, generational, copying, compaction.
- Virtual Memory: Paging, demand paging, working set.
- Cache Locality: Spatial & temporal locality, cache lines, false sharing.
- Ownership & RAII: Deterministic cleanup in C++/Rust.
- Memory Leaks: Unreleased heap memory.

8) PRODUCTION USE CASES

- Request Objects**: HTTP request, RPC contexts (short lived)
- Buffers**: Network buffers, I/O buffers, serialization (heap)
- Caches**: In-memory caches, LRU maps (long lived)
- Recursion / DFS**: Call stack depth limits; convert to iterative if needed
- Language Runtimes**: JVM heap, Go heap, VB heap, thread stacks
- Embedded Systems**: Small stacks, static alloc, no heap / custom pools
- Game Engines**: Frame allocators, arenas, object pools
- Database Systems**: Memory arenas, buffer pools, page caches
- Networking**: Connection pools, packet buffers, zero-copy
- Interop / FFI**: Pinning, ownership boundaries
- High-Perf Systems**: Allocation-free inner loops, ring buffers

9) INTERVIEW THINKING

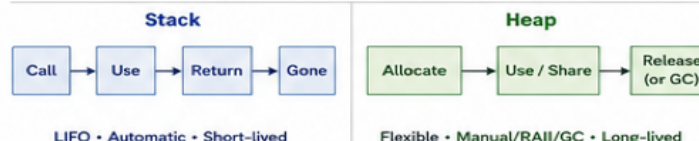
1. **Lifetime**: How long does the data need to live?
2. **Size**: How big is it (fixed, variable, potentially large)?
3. **Ownership**: Who owns it? Unique or shared?
4. **Language/Runtime**: GC? RAII? Borrow checker? Threads?
5. **Allocation Frequency**: Hot path or cold path?
6. **Locality**: Will it benefit from contiguous memory?
7. **Failure Mode**: Stack overflow vs. OOM / fragmentation?
8. **Cleanup**: Automatic (scope end/RAII/GC) or manual?
9. **Measure**: Profile allocations, retention, and pressure.
10. **Choose Representation**: value vs. reference, pools, arenas.

10) SENIOR NOTES

- ★ Stack vs. heap is about runtime placement, not semantic quality.
- ★ Compilers may optimize: inlining, SROA, stack allocation, elision.
- ★ Heap allocation cost includes future reclamation (GC or free). Always account for total cost of ownership.
- ★ Small object churn hurts GC and cache; prefer reuse/pools.
- ★ Pools / arenas reduce allocation cost and fragmentation at the expense of retained memory.
- ★ Respect alignment and padding; avoid false sharing.
- ★ Never return references/pointers to stack memory.
- ★ Prefer RAII / deterministic cleanup when possible.
- ★ Measure under production-like load; assumptions lie.

11) MENTAL HOOK

“The stack follows call lifetime; the heap follows data lifetime.”



Remember:

- Choose by lifetime
- Choose by size
- Choose by ownership
- Measure & validate
- Clean up everything

MEMORY ALLOCATION

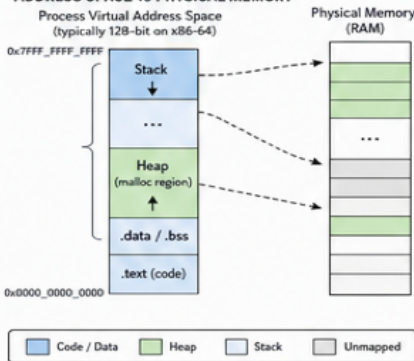
ENGINEERING MAP FOR SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEW PREP

1. WHY

Map variable lifetime/size requests onto finite virtual/physical memory efficiently and safely.

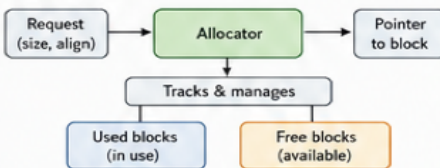
- Provide fast, scalable allocation & deallocation.
- Minimize fragmentation and RSS.
- Maximize locality and cache efficiency.
- Support concurrency and NUMA.
- Enforce alignment, isolation, and security.
- Fail predictably under memory pressure.

ADDRESS SPACE vs PHYSICAL MEMORY



2. CORE IDEA

An allocator returns a suitably sized and aligned memory block and tracks free/used memory. The allocation strategy balances speed, fragmentation, locality, and concurrency.



COMMON ALLOCATION STRATEGIES



SIZE CLASS EXAMPLE (64-bit, 16-byte alignment)

Class	Size Range (bytes)	Alignment	Used For
1	1 - 16	16	tiny objects
2	17 - 32	16	small structs
3	33 - 64	16	small/medium
4	65 - 128	16	medium
5	129 - 256	16	large small
6	257 - 512	16	large
7	513 - 1K	16	very large
...	...	...	...
Large	> 1K	page size	direct map

3. INTERNALS

1. **OS Interface:** obtain memory via `mmap()` / `brk()`. OS manages pages (e.g., 4KB) and page tables.
2. **Arenas:** allocator manages one or more arenas (heaps) per thread or NUMA node.
3. **Size Classes & Bins:** requests rounded to size class; each class has a bin/free list or slab.
4. **Free Lists:** doubly-linked lists (LIFO/FIFO) per bin. Fast path pops/pushes from list.
5. **Bump / Arena Allocators:** pointer "bumps" forward; reset or free en masse.
6. **Splitting & Coalescing:** split larger free blocks on alloc; coalesce adjacent free blocks on free() to reduce external fragmentation.
7. **Block Layout** (header optional):

Prev Size (optional)	Size	Flags	User Data (aligned)	Padding
Header		Payload		
		(alignment)		

 - Size usually includes header; flags: in-use, prev-in-use, etc.
 - Footer (rare) used by some allocators for coalescing.
8. **Alignment:** return pointer aligned to 2^k (8/16/32/64). Ensures correctness and enables vector/CAS.
9. **Thread Caches** (tcache/ magazines): per-thread small object caches to reduce contention.
10. **Large Objects:** above threshold (e.g., > 64KB) allocated via `mmap` directly; returned via `munmap`.
11. **Zeroing & Security:** optional memory zero on alloc/free; hardened metadata, canaries, quarantine.

4. COMPLEXITY

ALLOCATION / FREE COMPLEXITY (typical)

Allocator Type	Allocate	Free	Coalesce	Notes
Bump / Arena	$O(1)$	N/A	N/A	Reset $O(1)$
Segregated Fits (Bins / Slab)	$\sim O(1)$	$\sim O(1)$	$O(1)^*$	Fast path via lists
Buddy System	$O(\log N)$	$O(\log N)$	$O(\log N)$	Merge buddies
General (dmalloc/jemalloc)	Avg $\sim O(1)$ Worst $O(N)$	Avg $\sim O(1)$ Worst $O(N)$	$O(1)$ to $O(\log N)$	Size & bin dependent

* If immediate neighbors and sizes known.

OVERHEAD & FRAGMENTATION

Internal Fragmentation: Slack inside allocated blocks due to rounding, headers, alignment, or slab slots.

External Fragmentation: Free memory exists but not in a single contiguous block large enough.

Metadata Overhead: Headers, free list pointers, bitmaps, alignment padding.

Latency Distribution: Tail latency (p99/p999) matters more than average—measure it.

5. TRADE-OFFS

Approach	Pros	Cons
General-Purpose Allocator	Flexible size range Good avg. perf. Good fit for mixed workloads	More metadata Contention Fragmentation Complex internals
Pools / Slabs	Very fast Cache & TLB friendly Low fragmentation Predictable	Fixed sizes Retained memory Not memory tight
Arenas	Bulk free ($O(1)$) Minimal per-alloc cost Great for phases	Lifetime must align Memory blow-up if misused
Buddy System	Bounded fragmentation Simple coalescing Good for kernels	Power-of-two waste Internal frag. Higher overhead
Custom Allocator	Tailored to workload Can be optimal	Complex to build Hard to maintain / debug

No single allocator wins. Choose based on workload & constraints.

6. DISCUSSION (Be Ready)

- What is internal vs external fragmentation? How do you measure it?
- How does malloc (e.g., ptmalloc, jemalloc, tcmalloc) work internally?
- How does free() know the size of the block?
- Why is alignment important? What alignments are typical?
- What is an arena? When would you create more than one?
- What is a slab allocator? How does it differ from a pool?
- How does a buddy allocator split and coalesce blocks?
- How does realloc() work? When can it resize in place?
- How does an allocator achieve thread safety and scalability?
- How would you detect and debug memory corruption?
- What happens on out-of-memory (OOM)? What should the API do?
- How do allocators return memory to the OS?
- How do allocators handle large allocations?
- How would you design an allocator for a specific workload?
- What metrics would you instrument in production?

Key Terms: page, arena, bin, size class, slab, free list, coalesce, alignment, header, footer, fragmentation, RSS, TLB, NUMA, OOM.

Remember: Correctness > Performance > Memory > Simplicity.

7. RELATED TOPICS

Stack vs Heap	Stack: LIFO, fixed frames. Heap: dynamic, manual lifecycle.
Virtual Memory	Address spaces, page tables, demand paging, page faults.
Paging	Page size, TLB, page alignment and mapping costs.
Garbage Collection	Tracing, reference counting, generational GC.
Object Pools	Reuse to reduce allocation churn and fragmentation.
Cache Locality	Spatial/temporal locality; prefetching; false sharing.
NUMA	Local vs remote memory, first-touch, placement.
Ownership / RAII	Deterministic deallocation patterns in C++/Rust/Go.

8. PRODUCTION EXAMPLES

Runtime Heaps	malloc / new, JVM, Go runtime, Rust GlobalAlloc.
Databases	Arenas, buffer pools, page caches (e.g., InnoDB, PG).
Network Stacks	Packet buffers, sk_buffs, mbufs (DPDK/Netmap).
Game Engines	Frame allocators, transient pools, asset streaming.
Kernels	Slab allocators (SLUB/SLAB/SLOB), page allocators.
Embedded Systems	Fixed-size pools, no malloc, deterministic memory.
High-Frequency Trading	Lock-free pools, huge pages, predictable latency.
Serialization / IO	Buffers, arenas for parsing/encoding structures.
GPU / Accelerators	CUDA/HIP allocators, pinned memory pools.

9. INTERVIEW THINKING FRAMEWORK

1 Profile the workload	Sizes, lifetimes, allocation rate, peak/steady-state.
2 Define constraints	Alignment, latency SLOs, memory budget.
3 Concurrency & NUMA	Threads? Sharing? NUMA nodes? Placement?
4 Fragmentation bound	What % waste is acceptable? Internal vs external.
5 Reclamation strategy	Per-object free, bulk free, epochs, GC, refcount?
6 Failure & security	OOM handling, zeroing, guard pages, hardening.
7 Choose approach	General / pool / arena / slab / custom / OS mmap.
8 Instrument & iterate	Measure (p50/p99), tune, validate under load.

10. SENIOR NOTES

- ✓ Allocation cost includes: metadata, branching, atomics, cache misses, zeroing, TLB/page faults, and later reclamation.
- ✓ Group similar lifetimes. Phase your allocations. Bulk free whenever possible.
- ✓ Pools and caches can become leaks; set bounds and monitor high-water marks.
- ✓ Allocator contention often hides as tail latency. Per-thread caches help—measure cross-thread traffic.
- ✓ NUMA placement matters. Allocate on the right node; avoid remote accesses.
- ✓ Don't invent a general-purpose allocator casually. Use proven allocators unless you have a specific, measured need.
- ✓ Define and document the OOM contract clearly (return NULL, throw, abort?).
- ✓ Instrument everything: bytes in use, RSS, fragmentation, alloc/free rate, latency.

11. MENTAL HOOK

“An allocator trades metadata and policy to turn raw pages into useful lifetimes.”

GARBAGE COLLECTION

Senior Software Engineer / Technical Lead Interview Cheatsheet (2-3 min review)

1. WHY

- Automatically reclaim unreachable managed objects
- Prevent many use-after-free / double-free errors
- Simplify ownership & memory safety
- Free developers from manual deallocation in most cases



4. COMPLEXITY

- Work proportional to live data and allocated bytes depending on collector
 - Tracing: $\sim O(\text{live set} + \text{scanned roots})$
 - Reference counting: $O(1)$ per update but cycles problematic
- Allocation: usually amortized $O(1)$ (bump pointer)
- Pauses (latency): depends on algorithm, heap size, live set, concurrency
- Throughput: GC CPU vs application CPU
- Memory overhead: fragmentation, copying spaces, card tables, metadata
- Tail latency matters more than averages in many systems

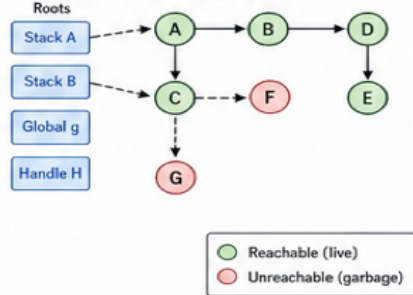
2. CORE IDEA

Roots (stacks, statics, globals, registers, handles) define reachability. Unreachable objects are garbage.

Approaches:

- Tracing (mark from roots)
- Reference counting (update counts)

Tiny Root Graph (Tracing)



3. INTERNALS

Collection algorithms

– Mark-Sweep



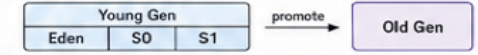
– Mark-Compact



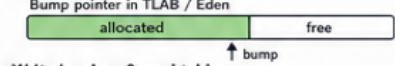
– Copying (Semi-space)



Generational collection



Allocation



Write barriers & card tables



Concurrency & pauses

- Stop-the-world (STW)
- Concurrent / Incremental / Mostly Concurrent

Safepoints

Threads reach safe points for STW or handoff

Finalizers / Weak Refs / Phantom Refs

Special handling: not for critical resources

GC Roots

Stacks, registers, globals/statics, JNI/handles, class metadata, VM internals

5. TRADE-OFFS

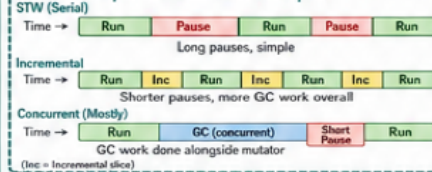
Pros	Cons
Memory safety, fewer bugs	Pauses (STW)
Simpler development	Extra CPU for GC
Compaction reduces fragmentation	Higher memory overhead
Generational = fast for short-lived objects	Nondeterministic reclamation
Concurrent collectors improve latency	More complex; harder to reason
Productivity	Not a silver bullet

Collector goals: maximize throughput, minimize latency, minimize footprint.
No one best collector—choose for workload.

6. DISCUSSION (Be Ready)

- How does GC identify garbage?
- How does GC handle cycles?
- What is the generational hypothesis?
- What is a write barrier and why needed?
- What is Stop-The-World (STW)?
- How do concurrent collectors work?
- How does compaction help? Costs?
- What are finalizers? Problems?
- What about weak/soft/phantom refs?
- How can memory leak still happen in a GC language?
- How do you tune GC / heap?

Simple Pause Timeline Examples



8. PRODUCTION CONTEXTS

JVM: G1, ZGC, Shenandoah Server apps, microservices	Game engines & runtimes
.NET CLR: Workstation / Server GC, LOH, Gen0/1/2	Low-latency trading services
Go: concurrent, non-moving GC, pacer, stacks shrink	Serverless / FaaS
JavaScript (V8, SpiderMonkey, JSC, Node.js)	Databases & caches
Python: Refcount + Cycle GC (generational)	Big data / stream processing
Mobile Apps (ART, iOS ARC + ObjC/Swift, Kotlin on ART)	Native interop (JNI/Plinvoke)
	Embedded & IoT
	Always consider native memory, off-heap, and resource lifetimes.

9. INTERVIEW THINKING FLOW

- Understand workload: allocation rate, object size dist, live set, mutation rate
- Define goals: latency SLOs, throughput, memory footprint, tail latency
- Pick runtime & collector candidates matching goals and ecosystem
- Size the heap: min/max, headroom, young/old ratios, region size
- Design object lifetime: short-lived vs long, ownership, reachability discipline
- Reduce churn & retention: pooling (careful), reuse, avoid accidental roots, weak refs
- Observe: pauses, GC CPU, alloc rate, live set, promotion rate, fragmentation
- Load test with production-like workloads (steady + spikes)
- Tune: flags, heap, threads, GC algorithm options
- Iterate: verify under load, watch for regressions; tune last, not first

10. SENIOR NOTES

- ✓ GC reclaims unreachable, not unused-by-business.
- ✓ Live set size drives GC cost more than heap size.
- ✓ Finalizers are not timely cleanup. Use try-with-resources / using / defer.
- ✓ Object pooling: can hurt (keeps objects alive, increases retention).
- ✓ Memory leaks still happen via reachable references (static caches, listeners, maps, closures, threads).
- ✓ Optimize allocation only with profiles. Fewer & shorter-lived objects > micro-opts.
- ✓ Container limits & cgroups affect GC. Right-size heap to avoid OOM kills.
- ✓ Always test worst-case & peak traffic. Tail latency matters.

11. MENTAL HOOK

GC follows reachability from roots; anything still reachable is alive whether useful or not.

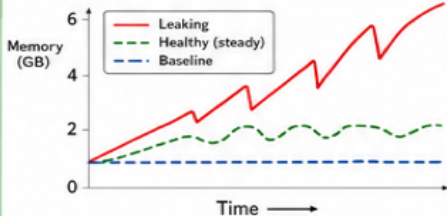
MEMORY LEAKS

SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEW PREP (2-3 MINUTE REVIEW)

1. WHY

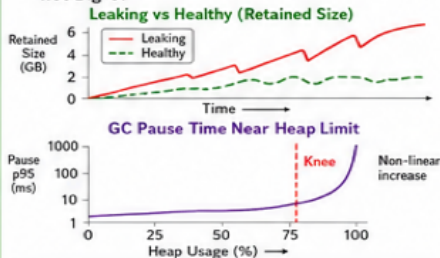
Retained memory/resources grow beyond intended lifetime, causing:

- GC pressure → more CPU, longer pauses
- Paging / swap → latency, thrash
- OOM (Out Of Memory) → crashes
- Latency spikes, timeouts, cascading failures, outages
- Wasted capacity and higher cost



4. COMPLEXITY

- Growth rate: bytes/time or bytes/request.
- Retained size vs shallow size: Retained includes everything reachable.
- Allocation rate vs live set: High allocation is OK if live set stable.
- GC cost (pause/CPU) grows non-linearly near heap limit.
- Detection uses slopes & diffs over time, not Big O.



Rule of Thumb

Sustained positive slope of retained size under steady workload = probable leak.

2. CORE IDEA



1) Native leak

Memory allocated in native heap (malloc/new) not freed or lost pointer → unreachable.



2) Managed leak

Object still reachable unintentionally (e.g., from a GC root path).



3) Resource leaks

OS/Runtime resources not closed: file descriptors, sockets, threads, handles.

Scenario	What grows?	Example	Fix
Leak	Retained size keeps growing	Unbounded Map, listener never removed	Fix retention / close
High legitimate usage	Live set grows to match load then plateaus	More users, larger cache (with bound)	Scale or optimize
Fragmentation	RSS high, live set flat	Many alloc/free size mismatch	Tuning / compaction

Leak = unintended growth of retained memory/resources.

3. INTERNALS



GC Root Retention

Roots (stacks, statics, registers, JNI, GC handles) → reference graph → retains objects.



Common Retention Sources

Static collections, singletons, event listeners, closures/captures, thread locals, caches.



Reference Cycles (Recounted Runtimes)

Cycles not collected without cycle detector (e.g., Python, PHP, some C++).



Native Ownership

C/C++ objects, buffers, arenas — ownership mismatch or never released.



Allocator & RSS Behavior

Freed memory may stay in process (arenas, per-thread caches, fragmentation).



Off-Heap / Direct Memory

Direct byte buffers, mmap, GPU memory, not visible in GC heap.



File Descriptor / Handle Leaks

Unclosed files, sockets, pipes, epoll/kqueue handles → EMFILE / resource exhaustion.

5. TRADE-OFFS



Caches / Pools

Improve latency & throughput by reusing objects/data but retain memory. Need explicit size/TTL budgets.



Weak / Soft References

Reduce retention but have subtle semantics and nondeterministic clearing.



Bounded vs Unbounded

Bounding (size/TTL/weight) avoids leaks but may increase miss rate/CPU.



Deterministic Cleanup

Using try-with-resources / defer / using / RAII prevents resource leaks; more code.



Restart

Masks leaks by resetting process state. May be acceptable short-term mitigation, not a real fix.

Optimize For

- Throughput
- Latency
- Memory
- Predictability
- Operability

Accept Trade-Offs

- More code/complexity
- Lower hit rate
- Slightly higher CPU
- Periodic evictions
- Restart disruption

6. DISCUSSION



Can a GC language still leak? How? (Yes—retention.)



Heap vs RSS? Why might RSS stay high after free?



How do you detect a leak in production?



How do heap dumps work? Dominators? Retention path?



How do you identify a native memory leak?



How do you debug a cache leak (unbounded growth)?



How do listener / callback leaks happen?



What about fragmentation? How to tell?



Are finalizers / destructors reliable for cleanup?



How do you reproduce a leak reliably?



How do you prove the fix? (Soak test, metrics, canary.)

7. RELATED



Garbage Collection

Algorithms, generations, GC roots, barriers, STW vs concurrent, compaction.



Allocators

jemalloc, tcmalloc, glibc malloc, arenas, per-thread caches, fragmentation.



Ownership / RAII

C++ RAII, Rust ownership/borrow checker, Go escape analysis.



Weak / Soft / Phantom Refs

Java Soft/Weak/Phantom, .NET WeakReference.



Caching Patterns

LRU, LFU, TTL, window-TinyLFU, size/weight bounds.



Virtual Memory

Paging, swap, overcommit, cgroups, limits.



Observability

Metrics, tracing, profiling, heap dumps, alerts on slopes & headroom.

8. PRODUCTION HOTSPOTS



Unbounded Maps / Sets / Lists

(keys grow with time).



High Cardinality Metrics / Labels

(metrics systems, logs, tags).



ClassLoader / Plugin Leaks

(static refs keep loaders alive).



HTTP / DB Connections

(connection pools misconfigured, not closed).



Goroutines / Threads

(leaked goroutines, non-daemon threads).



Large Objects / Buffers

(image caches, ByteArrayOutputStream).



Direct ByteBuffers / Off-Heap

(not released, cleaner not run).



ORM / Sessions / Transactions

(long-lived sessions, 1st level cache).



Event Subscriptions / Listeners

(not unsubscribed, strong refs).



Native Code / FFI

(C/C++ leaks, JNI global refs, arenas).

9. INTERVIEW THINKING (WORKFLOW)



1 Confirm Growth Under Steady Workload

Reproduce growth with constant load. Graph retained size & RSS.



2 Classify the Leak

Heap (GC objects) vs Native (RSS/off-heap) vs Resources (FDs, threads, sockets).



3 Correlate Allocation & Live Set

Look at allocation rate, live set, promotion rate, GC pauses, FD count, thread count.



4 Take Snapshots / Dumps

Heap dumps, profiler snapshots, or native heap (pmap/heaptrack). Compare diffs over time.



5 Analyze Retention Paths

Find dominators / shortest paths to GC roots. Spot unexpected long-lived references.



6 Build Minimal Reproducer

Reduce to smallest case that still leaks. Automate the repro.



7 Fix Ownership & Bound

Remove unintended references, close resources, add bounds/TTL, use weak refs if appropriate.



8 Validate & Guard

Soak test, watch slopes & headroom. Add alerts and regression tests.

Result: Stable live set, flat slopes, healthy headroom.

10. SENIOR NOTES

- ✓ A leak is a lifetime mismatch: something still owns the path to the object/resource.
- ✓ RSS may not go down after free—allocators keep arenas for reuse.
- ✓ Cardinality (keys, labels) is a frequent, hidden leak. Audit it.
- ✓ Caches and pools require explicit budgets, TTLs, and eviction policies.
- ✓ Cleanup paths must be idempotent, exception-safe, and verifiable.
- ✓ Heap dumps may contain PII/secrets. Handle securely and minimize retention.
- ✓ Monitor slopes and headroom, not just absolute usage or single datapoints.

11. MENTAL HOOK



A memory leak is data outliving its purpose because something still owns the path to it.



CACHE LOCALITY

ENGINEERING MAP FOR SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEWS

1. WHY

CPUs fetch memory in cache lines; access pattern often dominates runtime despite same Big O.

Cost per access (rough order of magnitude)

	L1 hit	~1 ns
	L2 hit	~4 ns
	L3 hit	~12 ns
	DRAM	~80–120 ns
	SSD	~50–150 µs
	Network	~0.1–1 ms

1000x+

Same $O(n)$ algorithm can be 10x–100x slower with poor locality.

2. CORE IDEA

Locality is the key to fast memory access.



Spatial Locality

Nearby addresses tend to be accessed again soon.
(Contiguous data, small strides)



Temporal Locality

The same data tends to be reused again soon.
(Loops, hot working set)



If your working set fits in cache, memory is fast. Otherwise, misses are expensive.

3. INTERNALS

- **Hierarchy:** L1 (32–128 KB) → L2 (256 KB–2 MB) → L3 (2–64 MB) → RAM (GBs)
- **Cache line:** typically 64 bytes (common).
- **Address split:** [TAG | INDEX (set) | OFFSET]

TAG	INDEX	OFFSET
(variable)	(e.g., 6–12 bits)	(e.g., 6 bits for 64B)

- **Set associative:** N-way (e.g., 8-way) within a set.

Set i

Way 0	Way 1	...	Way N-1
-------	-------	-----	---------

- **Hardware prefetchers** detect strides/sequences.
- **Row-major arrays:** last index changes fastest.
- **Pointer chasing** breaks spatial locality.
- **TLB locality:** fewer page walks are faster.
- **False sharing:** different cores, same cache line.
- **NUMA:** memory closer to the core is faster.

4. COMPLEXITY (BIG O UNCHANGED)

- **Big O** stays the same, but memory-access cost can differ by orders of magnitude.

- **Time** $\approx$ Instructions + Misses $\times$ Miss Penalty

Runtime $\approx$ Base + (Miss Rate $\times$ Miss Penalty)

- **Bandwidth $\neq$ Latency:** streaming uses bandwidth; random misses pay full latency.

- **Roofline intuition:** you're often memory-bound.

- **Blocked (tiled) algorithms** increase locality.

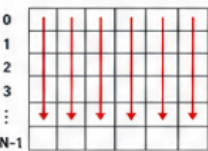
Example: Matrix ($N \times N$) Access (row-major storage)

Row-major traversal
(i, j): for $i=0..N-1$, for $j=0..N-1$



Access pattern: contiguous
Good locality (few misses)

Column-major traversal
(i, j): for $j=0..N-1$, for $i=0..N-1$



Access pattern: stride = N
Poor locality (many misses)

Both are $O(N^2)$, but column-major can be 10x–100x slower.

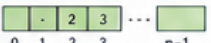
5. TRADE-OFFS

Improve Locality	Costs / Trade-offs
Contiguous data (vectors, arrays)	Reallocation/copies, insert/delete cost
Blocking / Tiling	More complex code, tile size tuning
SoA over AoS	Transformation cost, less ergonomic updates
Padding / Alignment	Extra memory footprint
Caching (software)	Staleness, coherency, invalidation complexity
Prefetching	May waste bandwidth or pollute cache

Pick the right trade-offs for the workload and keep hot paths simple.

6. DISCUSSION TOPICS

- What is locality? How does it differ from cache?
- What is a cache line? Why 64 bytes?
- AoS vs SoA: when and why?
- What is false sharing? How to fix it?
- What is the TLB? Why does it matter?
- Linked list vs vector: locality trade-offs?
- How does matrix blocking improve performance?
- How do hardware prefetchers work? Pitfalls?
- What is NUMA? How to write NUMA-friendly code?
- How would you measure cache misses?

Array / Vector	Linked List																		
Contiguous memory  • Good spatial locality • Cache-friendly • Predictable prefetch	Scattered nodes  VS. <table><tr><td>X:</td><td>x0</td><td>x1</td><td>x2</td><td>x3</td><td>...</td></tr><tr><td>Y:</td><td>y0</td><td>y1</td><td>y2</td><td>y3</td><td>...</td></tr><tr><td>Z:</td><td>z0</td><td>z1</td><td>z2</td><td>z3</td><td>...</td></tr></table> • Good for per-field scans, SIMD • Better cache utilization for scans	X:	x0	x1	x2	x3	...	Y:	y0	y1	y2	y3	...	Z:	z0	z1	z2	z3	...
X:	x0	x1	x2	x3	...														
Y:	y0	y1	y2	y3	...														
Z:	z0	z1	z2	z3	...														

7. RELATED CONCEPTS



CPU Cache



Arrays & Data Layout



Memory Allocation



Virtual Memory & Paging



SIMD / Vectorization



Data-Oriented Design



NUMA Architectures

Row-major
(Element $[i][j]$)

Address = Base +
 $((i \times N) + j) \times \text{sizeof}(T)$

Cache line (64B)

Holds $16 \times \text{int32}$
or $8 \times \text{int64}$
or $64 \times \text{char}$

Alignment

Align to 64B
for cache efficiency.

8. WHERE IT MATTERS (PRODUCTION)



Databases: columnar stores, vectorized execution



Game Engines: ECS layouts, hot component scans



ML/AI: tensors, minibatches, memory-bound ops



Networking: packet processing, ring buffers



Image/Video: convolution, filters, stencils



Analytics: large table scans, aggregations



Serialization: packing/unpacking contiguous buffers



Search Engines: inverted indexes, posting lists



Object Pools: contiguous allocation, reuse

Finance/Trading: low-latency, predictable access

9. INTERVIEW THINKING FLOW

- 1 Understand access pattern (sequential, random, stride?)
- 2 Choose data layout (AoS vs SoA, contiguous, aligned)
- 3 Estimate working-set size vs cache capacity
- 4 Analyze reuse distance (temporal locality window)
- 5 Make it sequential / block / batch operations
- 6 Consider concurrency (false sharing, partitioning)
- 7 Consider NUMA (placement, locality, pinning)
- 8 Plan measurement (counters, profilers, benchmarks)
- 9 Validate with profile & benchmarks on real data

10. SENIOR NOTES

- Same $O(n)$ can differ 10x–100x due to locality.
- Locality is workload- and data-dependent.
- Prefer contiguous data for hot scans & SIMD.

- Padding fixes false sharing but costs memory.
- Measure cache, TLB, and branch behavior together.
- Avoid pointer-rich abstractions in hot loops.

11. MENTAL HOOK

Cache locality makes future memory accesses cheap by making them predictable and nearby.

VIRTUAL MEMORY

Engineering Map for Senior Software Engineer / Technical Lead Interview Prep

1 WHY

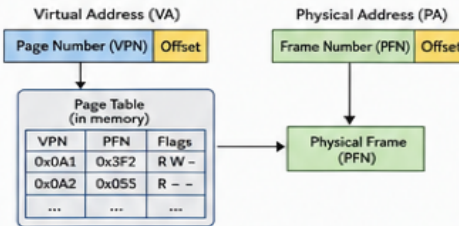
- Give each process a private, contiguous address space (simple model for programming and security).
- Isolation & protection between processes and kernel; controlled sharing.
- Map more memory than physically available; sparse address spaces.
- Enable features: sharing (libs, mmap), files, copy-on-write (fork), demand loading, overcommit.



Illusion of infinite, safe, private memory.

2 CORE IDEA

Virtual address $\rightarrow$ Page number + Offset
 $\rightarrow$ Page table maps to Physical frame; MMU enforces permissions.



$VA = (VPN \ll \text{PAGE_SHIFT}) \mid \text{OFFSET}$
 $PA = (PFN \ll \text{PAGE_SHIFT}) \mid \text{OFFSET}$

3 INTERNALS

- Multi-level page tables: e.g., 4-level (x86-64) PML4 $\rightarrow$ PDPT $\rightarrow$ PD $\rightarrow$ PT $\rightarrow$ Frame.
- TLB (Translation Lookaside Buffer): hardware cache of recent VA $\rightarrow$ PA translations.
- Page size: 4 KiB typical; also 2 MiB (Huge), 1 GiB (Giant).
- Demand paging: pages brought in on first access; page fault trap if not present.
- Backed types: Anonymous (heap/stack, zero-filled) vs File-backed (mmap, shared libs, executables).
- Copy-on-Write (COW): share read-only after fork; write triggers private copy.
- Protection bits: R, W, X, User/Supervisor, NX, Accessed (A), Dirty (D).
- ASLR: randomize layout for security.
- Swap: evict cold pages to disk (swap space).
- Huge pages: fewer TLB misses, larger allocation granularity.
- Kernel/User split: user space below, kernel space mapped high (or vice versa).

4 COMPLEXITY

Operation	Typical Latency (ns) (rough order of magnitude)
TLB hit	$\sim 1 - 2$
L1 hit (after TLB hit)	$\sim 1 - 4$
L2/L3 hit	$\sim 10 - 40$
DRAM access	$\sim 70 - 150$
Minor page fault (no disk I/O)	$\sim 1,000 - 10,000$
Major page fault (with disk I/O)	$\sim 100,000 - 10,000,000+$

- Translation usually fast on TLB hit.
- TLB miss $\rightarrow$ page table walk (memory reads).
- Page faults are 3-6 orders of magnitude slower.
- Page table memory overhead is real.
- Page size tradeoff: small = less internal fragmentation, more TLB misses; large = more waste, slower alloc.
- Working set that doesn't fit RAM $\rightarrow$ thrashing (continuous faults, low CPU utilization).

Effective Access Time (EAT)

$= P(\text{TLB hit}) \times T(\text{hit}) + P(\text{TLB miss}) \times (T(\text{walk}) + P(\text{fault}) \times T(\text{fault}))$

5 TRADE-OFFS

Gain	Cost
Isolation & protection	Translation overhead (TLB walks)
Simplicity (contiguous VA)	Page table memory
Flexibility (map anything)	Page faults (latency spikes)
Sharing (mmap, libs, COW)	Nondeterminism (faults, reclaim)
Overcommit & sparse address spaces	OOM risk at runtime
Huge pages: fewer TLB misses	More internal fragmentation; slower alloc; compaction needs
Demand paging	Requires backing store (disk); I/O costs

Virtual memory optimizes for average case performance and developer productivity; you pay in complexity and tail latency risk.

6 DISCUSSION (Interview Prompts)

- Virtual vs physical addresses—what's the difference?
- How does the TLB work and why is it important?
- What happens on a page fault?
- Minor vs major page faults—difference and cost?
- What is mmap()? When would you use it?
- How does copy-on-write work during fork()??
- How does the OS decide what to evict (replacement)?
- What is swap? When is it used?
- What are huge pages? When helpful vs harmful?
- How does segmentation differ from paging?
- Can two processes share memory safely?
- What is address space layout randomization (ASLR)?
- What happens when you run out of physical memory but still have free virtual addresses?
- Why can OOM occur even with free virtual space?

7 RELATED

- Paging: fixed-size pages map VA $\rightarrow$ physical frames.
- CPU Cache: TLB is to pages what cache is to lines.
- TLB: caches translations; separate I-TLB and D-TLB.
- Memory Allocators: malloc/new build on VM (arena, mmap, brk).
- Stack/Heap: grown on demand; guard pages protect.
- Filesystems & mmap: map files into address space.
- Processes & Context Switch: switch address space (idle or flush TLB depending on ASID/PCID).
- NUMA: locality across sockets; page placement matters.

8 PRODUCTION APPLICATIONS

- Process isolation & security (sandboxing, least privilege).
- Shared libraries (code + read-only data sharing).
- Memory-mapped databases & files (lazy I/O, zero-copy).
- Fork servers (COW to serve many children cheaply).
- Containers: share kernel, isolated via namespaces + cgroups (limits), not separate VMs.
- JIT & code pages: W*X policy, executable mappings.
- Zero-copy I/O: mmap + sendfile/splice for throughput.
- Large heaps: sparse address spaces, on-demand commit.
- Shared memory IPC (POSIX shm, memfd, tmpfs).
- Crash dumps: core files capture virtual address state.

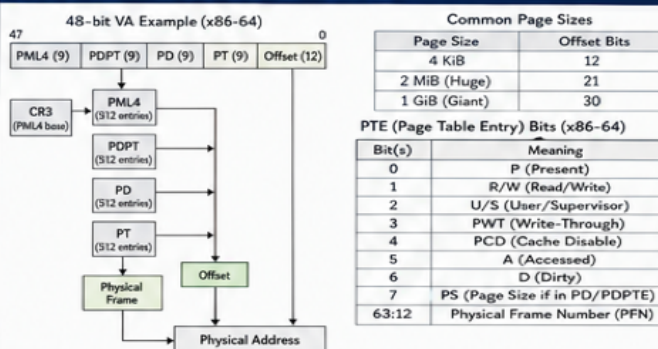
9 INTERVIEW THINKING PATTERN

- Start: What is the problem / requirement?
- Address space model: width, page size, user/kernel split.
- Map: build mental map of mappings & permissions.
- Locality & working set: will it fit in RAM?
- Faults & backing store: what happens on miss?
- Sharing & COW opportunities?
- Limits & policies: overcommit, ulimit, cgroups, RLIMIT_AS.
- Observe: metrics (RSS, VSS, faults, TLB misses). Tools: vmstat, perf, /proc/pid/*, mincore.
- Decide: mmap vs heap, page size (huge?), pre-faulting, pinning (mlock), NUMA policy.
- Validate: measure, iterate, watch tail latency.

10 SENIOR NOTES

- VSS (virtual size) is not RAM usage; it's just address space reserved/mapped.
- RSS (resident set size) is physical memory used by the process, but excludes shared pages counted elsewhere (careful with interpretation).
- Page faults can dominate tail latency—optimize working set and prefetch where appropriate.
- mmap I/O still faults: first access pulls pages from disk.
- Overcommit means allocation "succeeds" but may fail later on touch (OOM killer, SIGSEGV).
- Containers share the host kernel VM; cgroups enforce memory limits and OOM selection.
- Huge pages are not a silver bullet—measure; alignment and fragmentation matter.

ADDRESS TRANSLATION WALK (4-LEVEL EXAMPLE)



11 MENTAL HOOK

Virtual memory makes every process believe it owns an address space while the MMU negotiates reality page by page.

PAGING

How virtual pages become physical memory

1 WHY

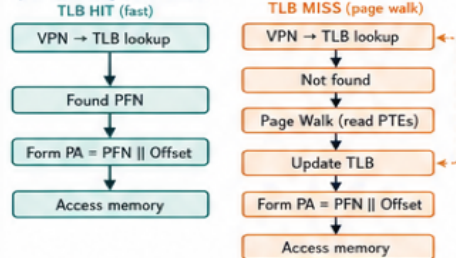
- Give each process a large, private, contiguous address space.
- Map only needed pages to physical frames (demand paging).
- Enable protection and isolation between processes.
- Efficient sharing (read-only, copy-on-write).
- Simplify allocation: fixed-size units; easier replacement and management.

KEY TERMS

 Virtual Page (VP)	Fixed-size chunk of virtual address space (e.g., 4 KiB).
 Physical Frame (PF)	Fixed-size chunk of physical memory.
 Page Table (PT)	Per-process structure mapping VPN → PF with permissions and status bits.
 Swap Space	Disk area used to store pages not in RAM.

4 COMPLEXITY

Address translation path (per memory reference)



Costs

- TLB hit: ~1–2 cycles.
- TLB miss → page walk: ~100s of cycles (4–5 memory references on x86-64 w/ 4-level page table).
- Page fault: ~10 μs (minor) to 10s of ms (major, I/O).

7 RELATED

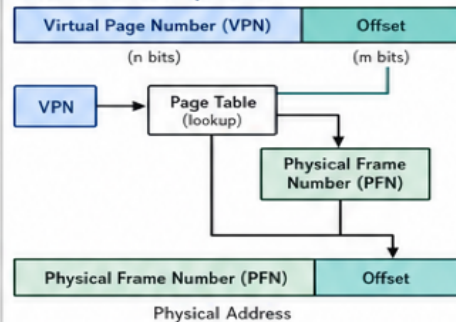
-  **Virtual Memory:** Paging is the core mechanism.
-  **TLB:** Cache of recent VPN → PFN translations.
-  **CPU Cache:** Caches data/instructions post-translation.
-  **Swap:** Backing store for evicted pages.
-  **mmap:** Map files/devices into address space.
-  **Allocators:** malloc/new use VM (brk/mmap) under the hood.
-  **NUMA:** Placement and locality affect performance.

10 SENIOR NOTES

- ✓ **Locality is king:** good locality → fewer TL misses/faults.
- ✓ **Right-size the working set:** fit hot set in RAM; monitor WSS (working set size).
- ✓ **Minimize tail latency:** avoid major faults in the hot path; pre-fault or warm where needed.
- ✓ **Use huge pages** where appropriate; measure fragmentation and allocation latency.
- ✓ **Avoid global TLB shootdowns;** batch changes; use PCID/ASID.
- ✓ **Observe and iterate:** track faults/sec, WSS, swap in/out, TLB miss rate, page cache hit rate.

2 CORE IDEA

Virtual address is split into:



VPN = Virtual Page Number PFN = Physical Frame Number
m = page offset bits (determined by page size)

5 TRADE-OFFS

Benefit	Cost
No external fragmentation	Internal fragmentation (average ~1/2 page waste)
Simple allocation (fixed-size units)	Page table memory overhead
Protection & isolation	Translation overhead (TLB + page walk)
Demand paging (saves RAM)	Page-fault latency and I/O cost
Sharing, COW, shared libs	Complexity (PTE bits, refcounts, shootdowns)
Huge pages reduce TLB misses	May increase internal fragmentation

Effective Access Time (EAT)

$$EAT = h \times (t_{mem}) + (1 - h) \times (t_{walk} + t_{mem})$$

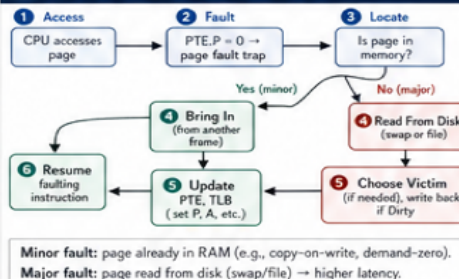
plus page-faults (if any)

h: TLB hit rate t_{mem} : memory access time
 t_{walk} : page-walk time (on TLB miss)

8 PRODUCTION

-  **OS Virtual Memory:** On-demand paging, sharing, COW, page cache.
-  **Databases:** mmap data files; OS handles paging and caching.
-  **Large Heaps:** Gigabytes via mmap; sparse pages charged only when used.
-  **Containers/Ggroups:** Memory limits; OOM killer; page accounting.
-  **Shared Libraries:** Mapped read-only; shared across processes.
-  **Zero-Copy I/O:** mmap + sendfile/splice; avoid extra copies.

11 PAGE FAULT LIFECYCLE



3 INTERNALS

Page Table Entry (PTE) format (typical)

63	(PFN)	11	10	9	8	7	6	5	3	2	1	0
Physical Frame Number		P	R/W	U/S	A	D	COW	NX	...			
P		Present (valid in memory)										
R/W		Read/Write (or permissions)										
U/S		User/Supervisor										
A		Accessed (set by hardware on read/write)										
D		Dirty (set by hardware on write)										
COW		Copy-On-Write										
NX		No-Execute										
...		OS-specific bits (e.g., global, swapped, protection)										

Fixed sizes

- Page size = 2^m bytes (e.g., 4 KiB → m = 12 bits)
- Frames same size as pages.
- Eliminates external fragmentation; may introduce internal fragmentation (last page).

6 DISCUSSION

-  **Page size:** Too small → more overhead; too large → more internal fragmentation, fewer pages in memory.
-  **Replacement policy:** LRU, Clock, Second Chance, ARC; balance hit rate vs overhead.
-  **Dirty pages:** Write-back improves throughput but needs flushing on eviction or crash.
-  **TLB:** Size, associativity, page coloring, shootdowns, PCID/ASID reduce flush cost.
-  **Huge pages:** 2 MiB/1 GiB reduce TLB misses; watch fragmentation and allocation latency.
-  **Thrashing:** Too many faults; CPU busy paging. Mitigate with working-set control and limits.
-  **Working set:** Keep active pages in RAM; size mem to working set + headroom.

9 INTERVIEW THINKING

1. Clarify: page size, address bits, architecture (32/64-bit).
2. Draw: VA split (VPN | offset) and the mapping flow.
3. Explain TLB: hit vs miss vs page walk.
4. Explain faults: minor (in RAM) vs major (from disk).
5. Discuss PTE bits and roles (P, A, D, perms, COW).
6. Analyze trade-offs: overheads vs benefits.
7. Consider design choices: page size, huge pages, replacement, dirty handling.
8. Connect to system impacts: locality, working set, throughput, tail latency.

MENTAL HOOK



Paging trades contiguous physical memory for fixed-size mappings—and makes locality determine whether the illusion is fast.

CPU CACHE

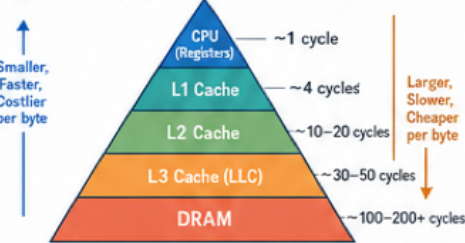
The hierarchy that hides memory latency

1 WHY

Bridge the CPU-DRAM latency gap. Exploit locality so most accesses hit in fast, small memories.

- **Temporal locality:** reuse recently used data.
- **Spatial locality:** tend to access nearby data.

Typical latency (cycles)



2 CORE IDEA

- Data moves in fixed-size blocks called cache lines (e.g., 64 B).
- On a miss, the whole line is fetched.
- Organized into sets and ways (associativity).
- Replacement picks a victim on a miss.
- Multi-level hierarchy (L1/L2/L3) filters requests before DRAM.

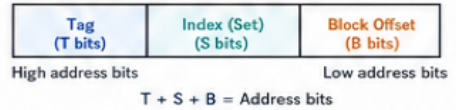


Hit: data found at that level.

Miss: fetch from next level (slower).

3 INTERNALS

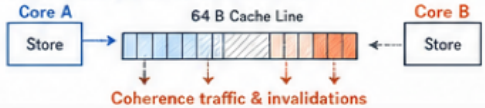
Address breakdown of a cache line



Set-associative lookup (E-way)



False sharing (two cores, same line)



4 COMPLEXITY

Levels

- L1 (split I/D): smallest, fastest, private.
- L2: larger, private.
- L3 (LLC): shared by cores.

Organizing a cache

- Line size: 32-128 B (common 64 B).
- Sets = (Cache Size) / (Line Size x Ways)
- Ways (E): 1 (direct-mapped) → higher associativity reduces conflict misses.

Replacement

- LRU (true or pseudo), FIFO, Random.

Write policies

- Write-Through (WT) vs Write-Back (WB).
- Write-Allocate vs No-Write-Allocate.

Prefetch

- Hardware prefetcher detects patterns and fetches lines early.

Inclusion

- Inclusive (L3 superset of L1/L2) vs Exclusive (no duplicates across levels).

Coherence (multi-core)

- MESI protocol: Modified, Exclusive, Shared, Invalid. Maintains a consistent view across caches.

5 TRADE-OFFS

	Benefit	Cost / Risk
	Transparent speedup (hits are fast)	Nondeterministic timing (misses are slow), hurts real-time predictability
	High performance per watt (on hit)	Silicon area & leakage power increase
	Reduces DRAM bandwidth pressure	Coherence traffic & interconnect contention
	Scalable hierarchy filters requests	Design complexity, verification, tuning effort
	Good locality → big wins	Poor locality → cache pollution, thrash, misses
	Prefetch hides latency	Wasted bandwidth, pollution if mispredicted

6 DISCUSSION

- How does line size affect locality, bandwidth, and false sharing?
- What associativity and replacement policy would you choose and why?
- When prefer write-back vs write-through? Write-allocate vs no-write-allocate?
- How do inclusive vs exclusive LLC designs trade off?
- How does coherence (MESI) work at a conceptual level?
- What causes false sharing and how do you fix it?
- How can data layout improve cache locality?
- When would you use cache-oblivious algorithms or layouts?

7 RELATED

- **Cache locality:** temporal & spatial locality, working set, reuse distance.
- **TLB:** caches virtual → physical translations; TLB misses are expensive.
- **Virtual memory:** pages (4 KB); page faults are far slower than cache misses.
- **Memory model:** ordering, fences, and visibility across cores.
- **SIMD/Vectorization:** wider operations, better cache line utilization.
- **NUMA:** memory latency/bandwidth vary by node; locality matters.

8 PRODUCTION

- **Databases**
Buffer pools, index locality, columnar layouts, avoid random I/O & pointer chasing.
- **Packet Processing**
Flow tables in cache, ring buffers, prefetch, per-core sharding to reduce sharing.
- **ML Kernels**
Tiling/blocking, weight reuse, layout (NHWC vs NCHW), prefetch, avoid cache thrash.
- **Game Engines**
Spatial partitioning, SoA layouts, streaming assets, reduce cache misses in hot loops.
- **High-Throughput Services**
Object pools, batching, lock-free queues, careful data placement and padding.

9 INTERVIEW THINKING

- 1 **Clarify the scenario**
Workload, access patterns, data sizes, core count.
- 2 **Identify likely bottlenecks**
High miss rate? Bandwidth saturated? Coherence?
- 3 **Form a hypothesis**
E.g., poor locality, false sharing, small cache, etc.
- 4 **Reason about cache behavior**
Estimate working set vs cache, reuse distance, strides, sharing patterns.
- 5 **Propose targeted changes**
Data layout, tiling, prefetch, padding, alignment, partitioning, algorithmic changes.
- 6 **Evaluate trade-offs**
Performance vs memory, power, complexity, latency determinism.
- 7 **Validate & measure**
Choose metrics, run experiments, iterate.

10 SENIOR NOTES

- ✓ Measure first. Trust counters, not intuition.
- ✓ Know your working sets at each level.
- ✓ Optimize data layout: SoA vs AoS, alignment, structure size, hot/cold splitting.
- ✓ Avoid pointer chasing: prefer contiguous arrays.
- ✓ Pad or align only when proven (e.g., false sharing).
- ✓ Use perf/PMU: LLC miss, L1 miss, load/store misses, coherence events, bandwidth.
- ✓ Think end-to-end: CPU, memory, interconnect, NUMA.
- ✓ Optimize for p50 and tail latency (misses hurt tails).

11 MENTAL HOOK



A CPU cache bets that the next useful byte is near one you just touched.